

CP020003

ARTIFICIAL INTELLIGENCE



Khon Kaen University · 2026

Week 05

Unsupervised Learning

Finding the Customers **Hiding** in Your Data

Clustering · RFM · PCA · K-Means · DBSCAN

— from raw transactions to profit-driving customer personas



Instructor: Teerapong Panboonyuen (P'Kao)

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1

You just got hired as the store's first data scientist

There's no column called Customer_Segment in the database. Nobody ever sat down and tagged a transaction “VIP” or “Budget Shopper.” All you have is a pile of raw receipts — and one real question from the CEO:

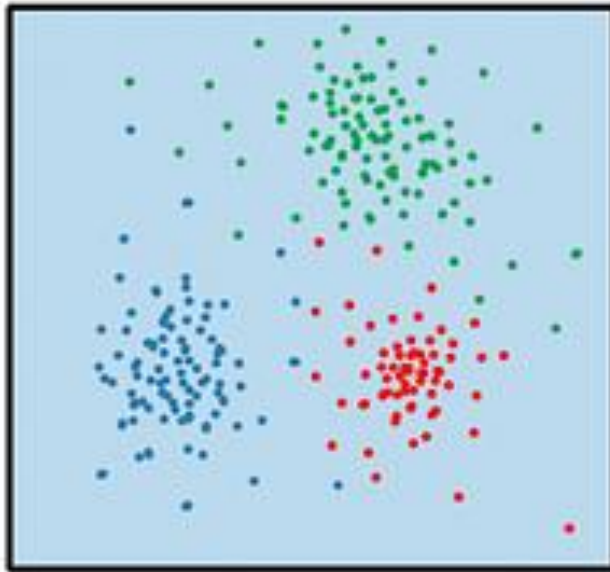
“What kinds of customers do we actually have — and what should we do about each one?”

For 4 weeks you had an answer key. This week, it disappears. That is the entire game of Unsupervised Learning: discover structure with zero labels, then turn it into a decision worth money.

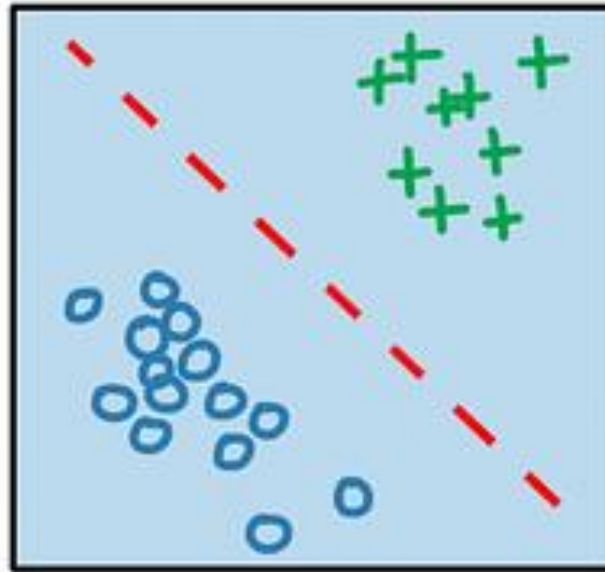
- 01 Raw Transactions**
Who bought what, when, how much
- 02 Hidden Structure**
Clustering finds the natural groups
- 03 Business Story**
Turn each group into a persona
- 04 Profit Action**
Campaigns that make money

machine learning

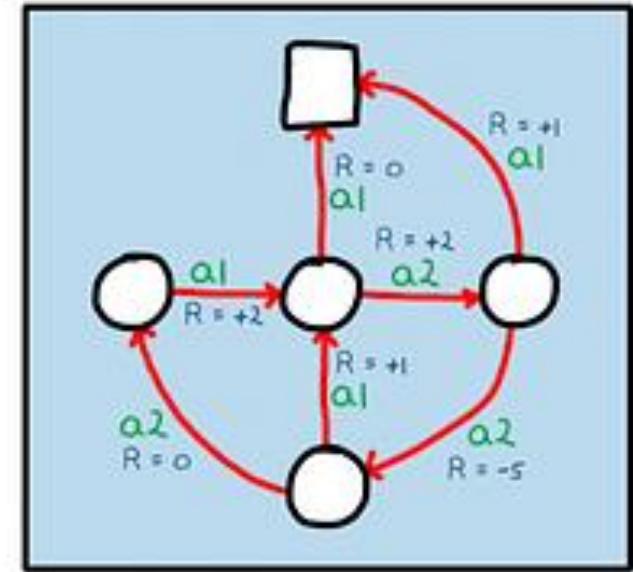
unsupervised
learning

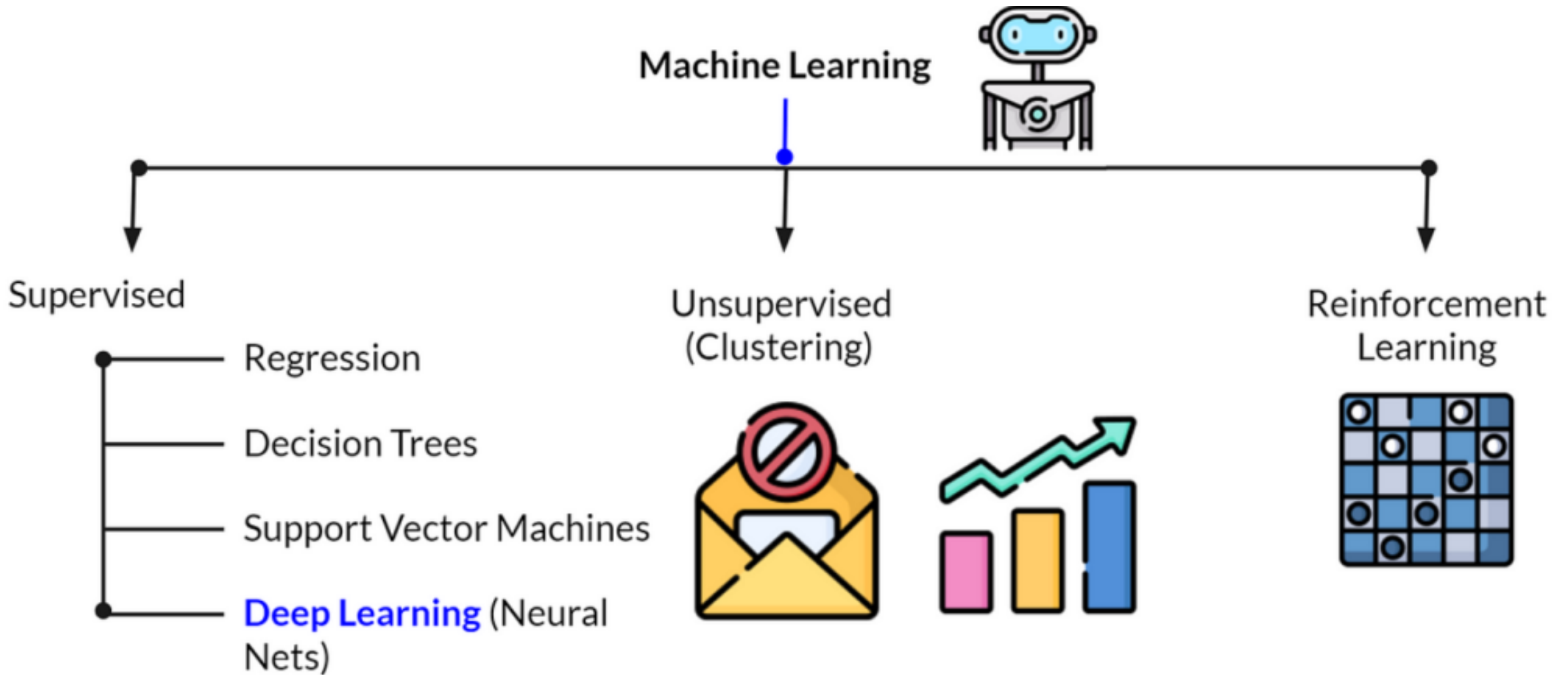


supervised
learning

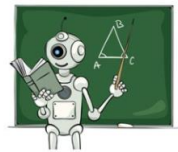


reinforcement
learning





Data in Supervised vs. Unsupervised Learning



Supervised Learning

VS



Unsupervised Learning

VS



Reinforcement Learning

Supervised Learning



Labeled Data

Unsupervised Learning



Unlabeled Data

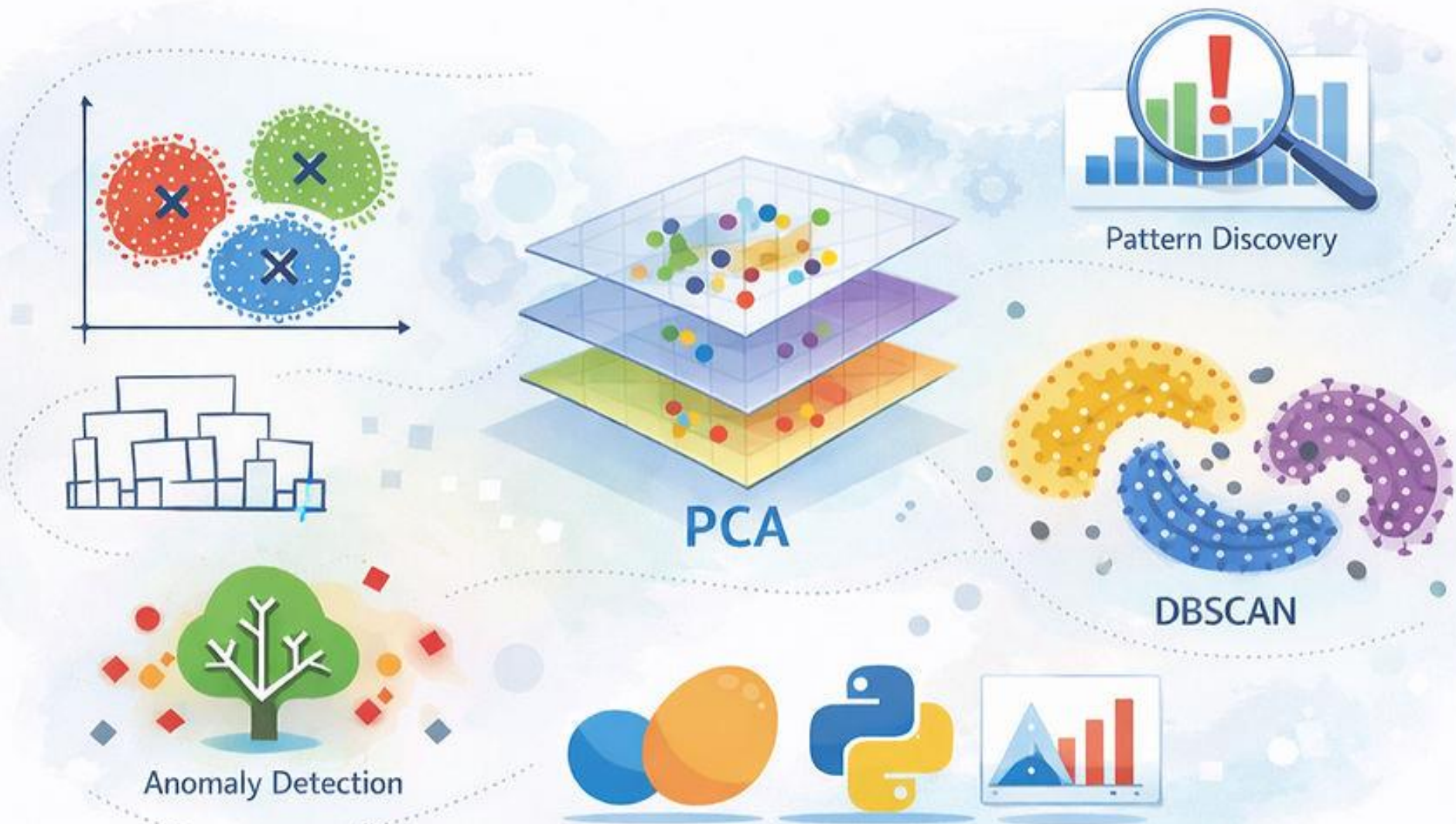
Hybrid Model that
Includes Supervised
Learning



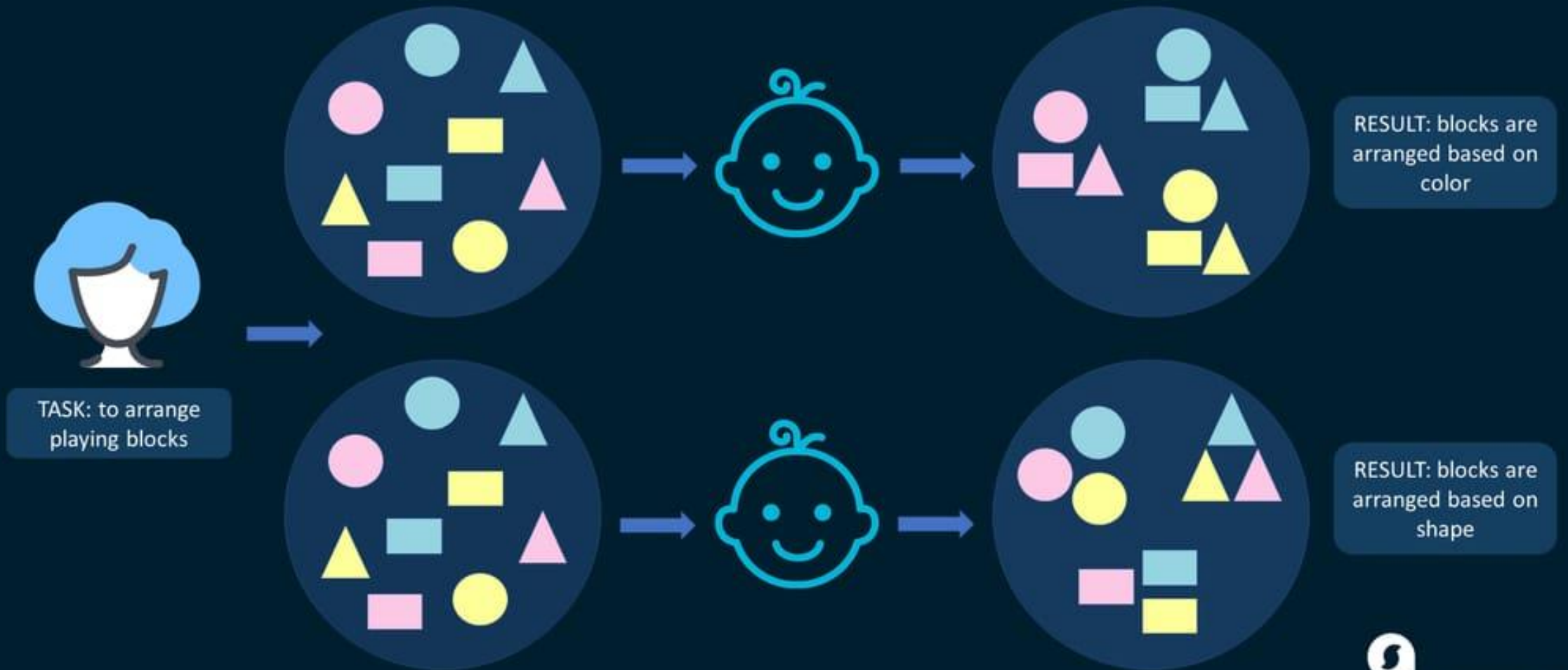
Labeled Data

&

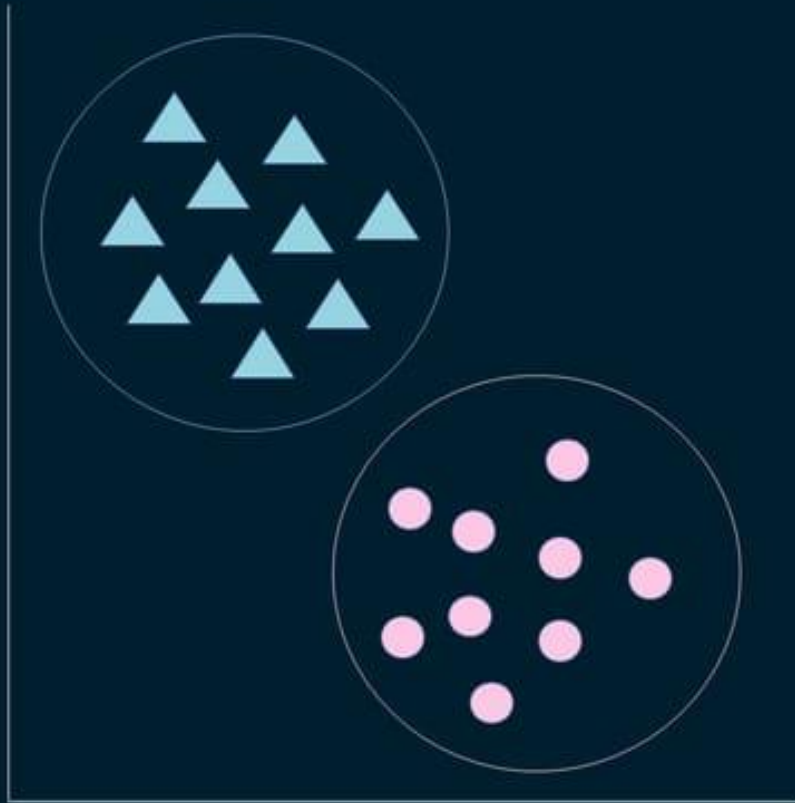
Unlabeled Data



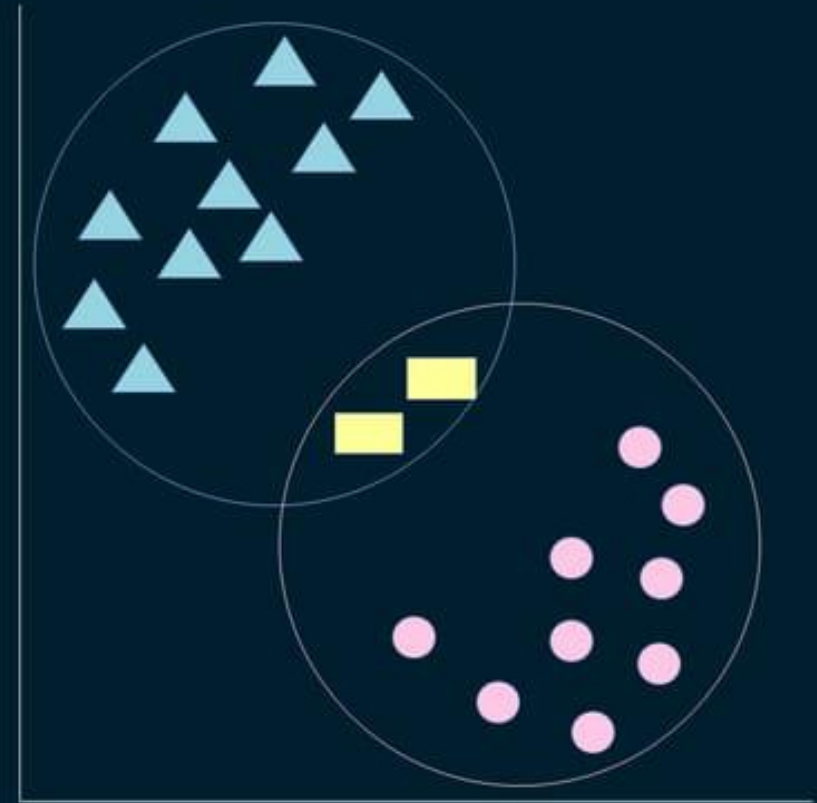
THE WORKING PRINCIPLE OF CLUSTERING



EXCLUSIVE VS OVERLAPPING CLUSTERING



Exclusive clustering



Overlapping clustering

Supervised vs. Unsupervised — What Changed?

	SUPERVISED (Weeks 1–4)	UNSUPERVISED (This Week)
Data	Features + known labels (y)	Features only — no y at all
Goal	Predict a known target accurately	Discover hidden structure & groups
Question	“Is this transaction fraud?”	“What kinds of customers exist?”
Retail use	Churn, demand, price sensitivity	Segmentation, anomaly detection
Success is...	High accuracy / F1 on held-out labels	Groups that are coherent AND useful

Retail reality: no retailer's database has a column called Customer_Segment. Unsupervised learning discovers those groups — and translating a cluster into a story and an action is the skill that pays.

What Is Clustering, Really?

Clustering groups data points so that points in the same group are similar to each other, and different from points in other groups — with nobody telling the algorithm what the groups are in advance.

- **No teacher**

The algorithm never sees a correct answer — only the raw feature space.

- **Similarity is geometric**

Points close together in feature space (after scaling!) get grouped together.

- **You supply meaning**

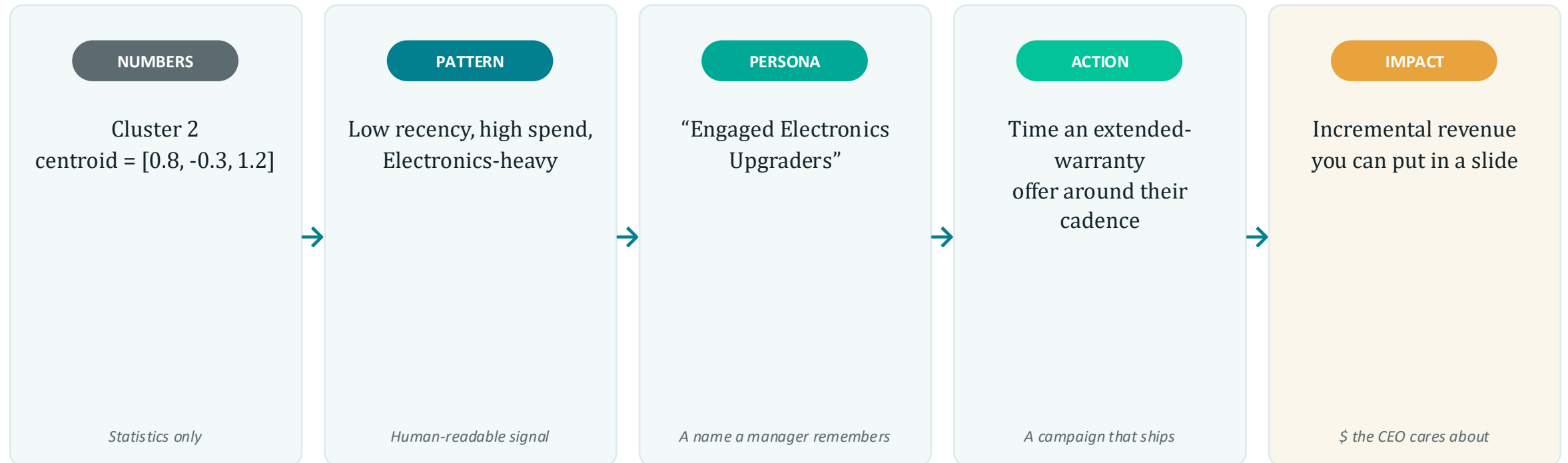
The algorithm finds Cluster 0, 1, 2, 3 — you decide what each one means for the business.



Same points. The only thing that changed is that we can now see — and name — the structure.

Clustering Only Pays Off If You Tell the Story

Going from “Cluster 2 has centroid [0.8, -0.3, 1.2]” to “these are our highest-value loyal shoppers — protect them with a VIP program” is the single most important, most-often-skipped skill in applied clustering.



Meet the Data: 1,000 Real Retail Transactions

Retail Sales Transactions (Kaggle, mohammadtalib786) — every row is one purchase, with who bought it, what, when, and for how much.

1,000

Transactions

9

Raw columns

3

Product categories

18–64

Customer age range

Columns: Transaction ID · Date · Customer ID · Gender · Age · Product Category · Quantity · Price per Unit · Total Amount

Three early trends worth noticing before we model anything:

Electronics is the high-roller

Widest spread & highest median spend (think \$500 items)
— Beauty is low-ticket, high-frequency.

Revenue $\neq$ transaction volume

A month can have more transactions but lower revenue, or fewer but higher — you need both signals, which is exactly the RFM idea.

Age barely predicts spend

Total Amount $\approx$ Quantity $\times$ Price — but Age is nearly uncorrelated with spending. “Older = bigger spender” is a myth here.

R-F-M: The Three Numbers That Explain a Customer

Before any algorithm runs, the classic RFM framework turns messy transaction history into three business-obvious signals per customer.



Recency

Days between a customer's most recent Order Date and a fixed reference date (max Order Date + 1 day).

Business question:

Are they still around?



Frequency

Number of distinct orders that customer has placed.

Business question:

Are they loyal, or a one-timer?

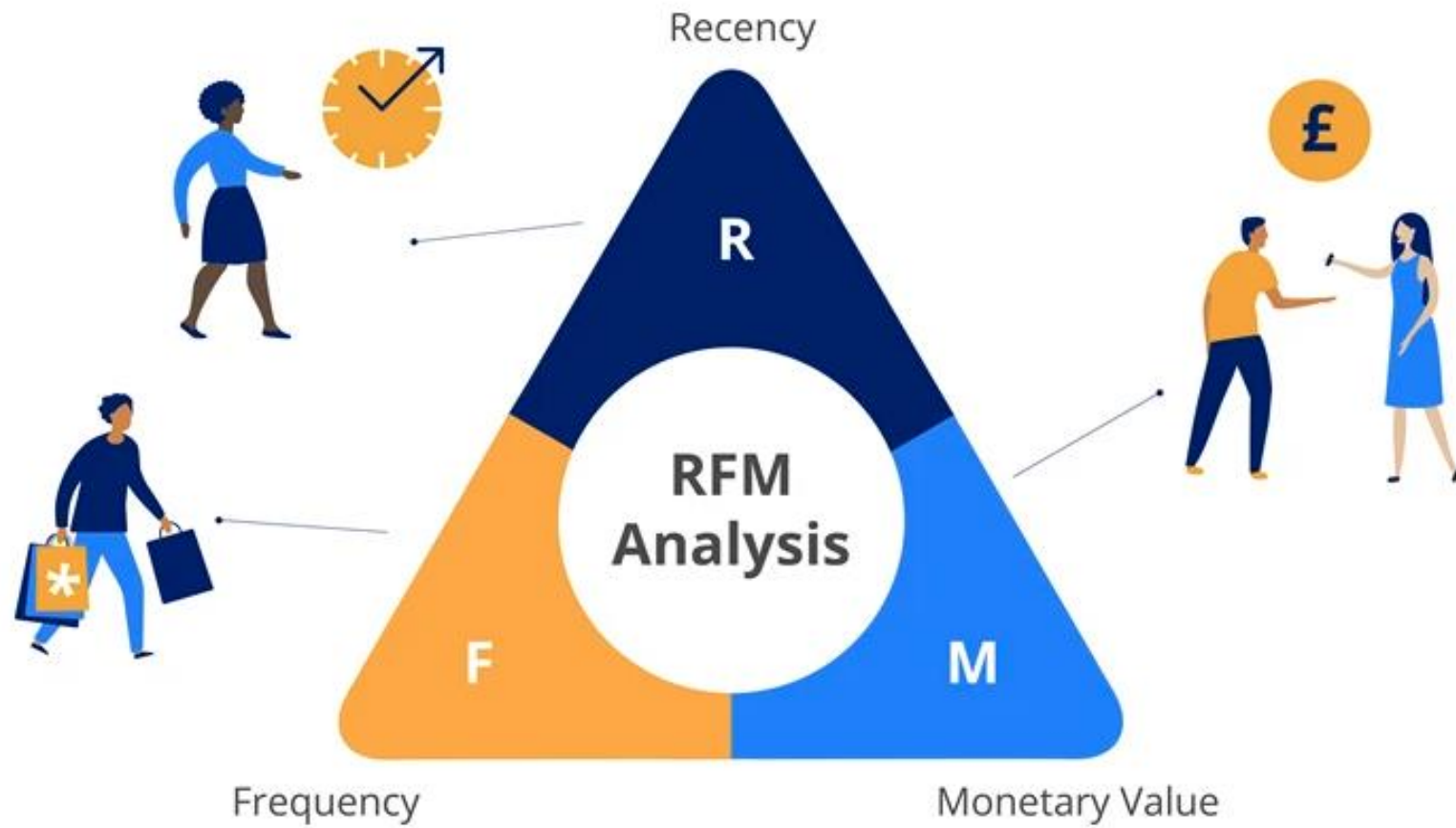


Monetary

Total spend by that customer — sum of Order Total across all their orders.

Business question:

How much are they worth?



Building the Feature Vector — An Honest Detour

Clustering algorithms don't group “transactions” in the abstract — they group whatever feature vector you hand them. Choosing that vector well is 80% of the job.

⚠ A real, common data limitation

In this dataset every Customer ID appears exactly once — no repeat purchases. So true Frequency is 1 for everybody, and we can't build the full classic RFM here.

We don't hide that — we build the closest honest equivalent: a behavioral feature set from what one transaction does tell us.

Production version (real repeat customers):

```
rfm = df.groupby("Customer ID").agg(  
    Recency=(...), Frequency=(...count),  
    Monetary=("Total Amount", "sum")
```

Our final 10-feature vector, per customer

- Recency — days since this purchase
- Monetary — total spend (Total Amount)
- Quantity — items in the basket
- AvgItemPrice — spend ÷ quantity
- Age — customer age
- Category flags × 3 (Beauty / Clothing / Electronics, one-hot)
- Gender flags × 2 (Male / Female, one-hot)

Scales already look suspicious: Monetary reaches into the thousands, Age is 18–64, one-hot flags are just 0/1. That mismatch is exactly our cue for the next section.

Feature Scaling — Why K-Means Cannot Live Without It

K-Means (and DBSCAN, and almost every distance-based algorithm) decides which points belong together using Euclidean distance:

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

The problem: this formula has no idea Monetary is dollars (range ~0–2000) while Gender_Male is just 0 or 1. Whichever feature has the largest numeric range dominates the distance — not because it's more important, but purely because of its units.

StandardScaler fixes this

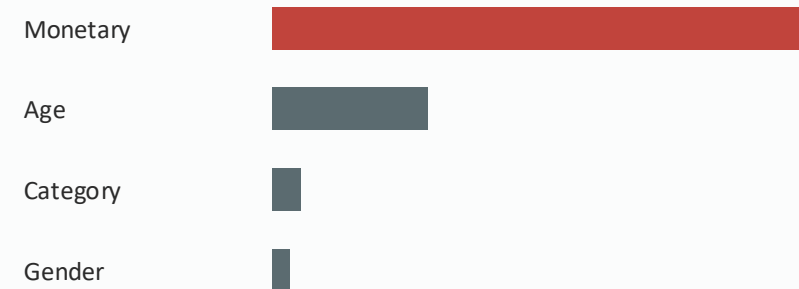
$$z_i = (x_i - \mu_i) / \sigma_i \quad \text{— mean} = 0, \text{std} = 1 \text{ for every feature}$$

After scaling, a \$50 spending gap and a 0.5-std age gap are treated as comparably meaningful — which is what we actually want.

Lesson: always scale before distance-based clustering — it silently changes which customers get grouped together.

Whose “vote” dominates the distance?

UNSCALED DATA



SCALED DATA (StandardScaler)



The Curse of Dimensionality

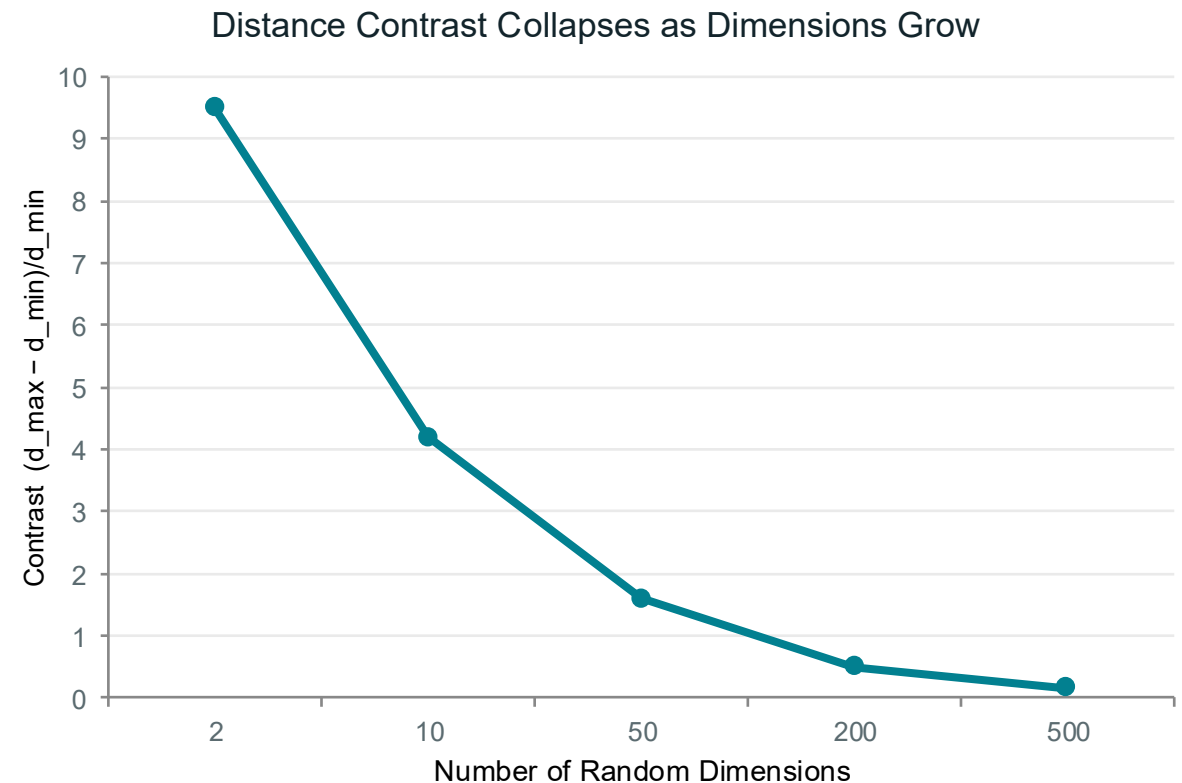
As we add more features, something strange happens to distance itself: in very high dimensions, all points start to look roughly equally far apart. “Near” and “far” — the entire idea clustering depends on — breaks down.

$$\text{Contrast} = (d_{\text{max}} - d_{\text{min}}) / d_{\text{min}}$$

High contrast = near/far are meaningfully different → good for clustering.

Contrast → 0 = every point is about the same distance from every other → no signal left.

Why it matters for us: our engineered set has only 10 dimensions — safe for now. But one-hot-encoding hundreds of SKUs, store locations, or browsing embeddings can push a real system into the hundreds or thousands — exactly where raw K-Means/DBSCAN starts to struggle.



Illustrative pattern from the notebook's random-point experiment — dramatic contrast at 2 dims, collapsing toward zero by 200–500.

PCA — Fighting the Curse, and Seeing Your Data

Principal Component Analysis finds new axes (principal components) that capture the maximum possible variance, ordered from most to least important. It must run on scaled data — otherwise, just like K-Means, it's dominated by whichever raw feature has the largest range.

1

Fights the curse of dimensionality

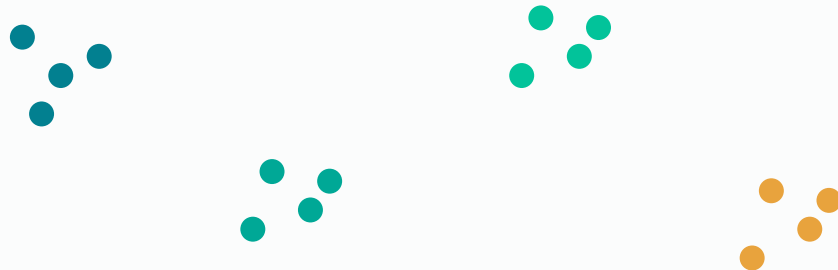
Keep only the components that carry real signal; drop the noisy, low-variance ones before distance-based clustering ever runs.

2

Free visualization

Even if we clustered on 10 (or 100) dimensions, we can always look at the result by projecting onto the first 2 principal components.

PC2

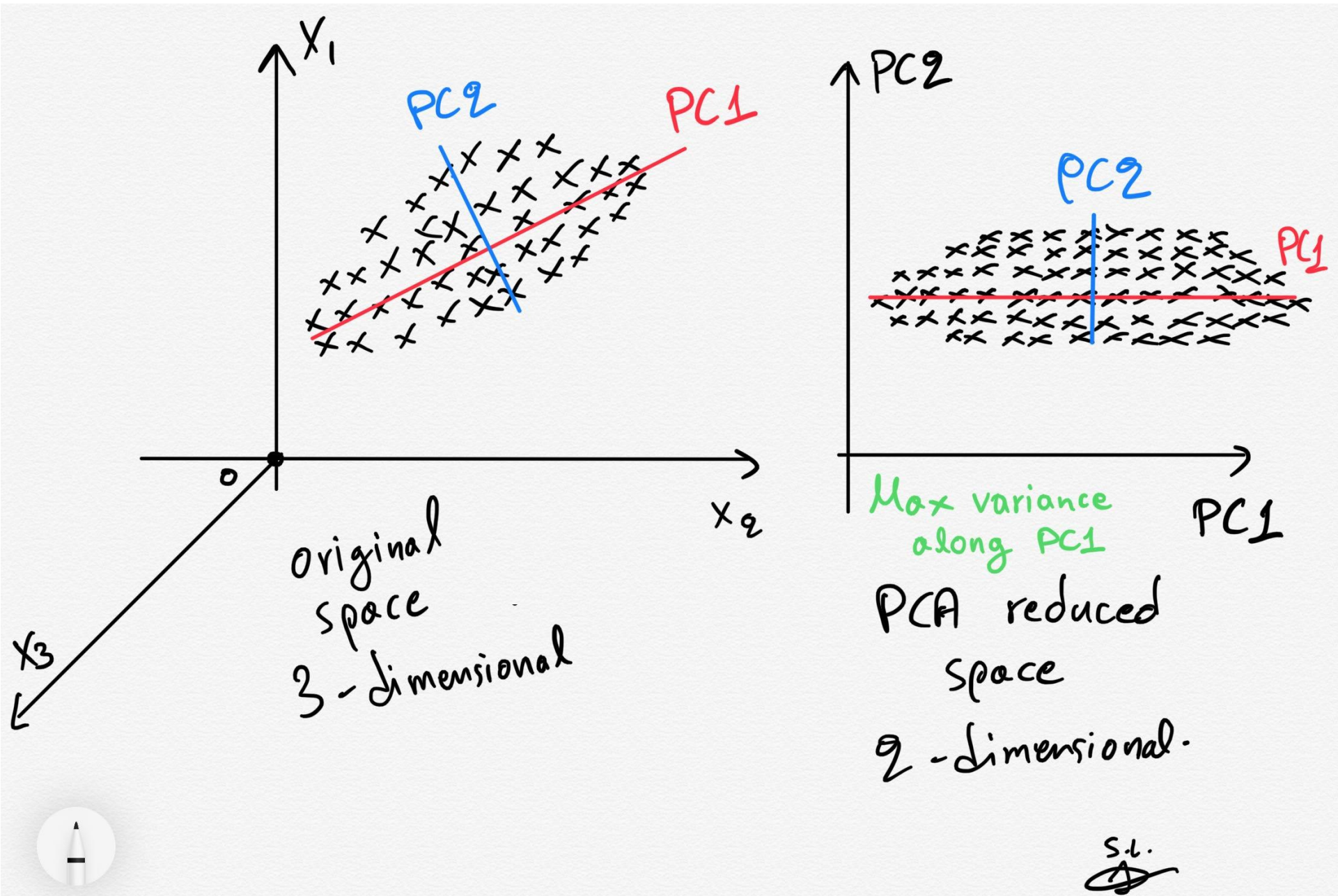


Order matters

PC1 always explains the most variance, PC2 the next most (orthogonal to PC1), and so on. Component 1 + 2 alone often capture most of what matters — that's why a 2D PCA plot is such an honest visual summary of a 10-D clustering result.

Even before clustering runs, a 2D PCA projection often already hints at structure — a sanity check to eyeball every algorithm's output against.

PC1



How Many Clusters? Four Metrics, One Decision

K-Means requires you to specify K in advance — but the business rarely knows if there are 3 segments or 7. We compute four complementary metrics across $K = 2..10$.

Inertia

Elbow Method

Sum of squared distances to centroid

Find the “elbow” — more K stops helping

Silhouette

Score (−1 to 1)

How much closer a point is to its own cluster vs. the next-nearest

Higher is better

Davies–Bouldin

Index

Average similarity between each cluster and its most-similar neighbor

Lower is better

Calinski–Harabasz

Index

Ratio of between-cluster to within-cluster dispersion

Higher is better

No single metric is perfect — that's exactly why we look at all four together, and let business sense break ties.

The Judgment Call: Why We Chose $K = 4$

The honest complication

Silhouette and **Davies-Bouldin** both technically prefer $K=10$, while **Calinski-Harabasz** prefers $K=2$.

The four metrics do not agree — because each measures a different notion of “good clustering,” and none know what a marketing team can actually use.

Why not $K=10$ or $K=2$?

$K = 10$ — segments too small & numerous for a human team to build separate campaigns around.

$K = 2$ — throws away almost all nuance between customers.

4

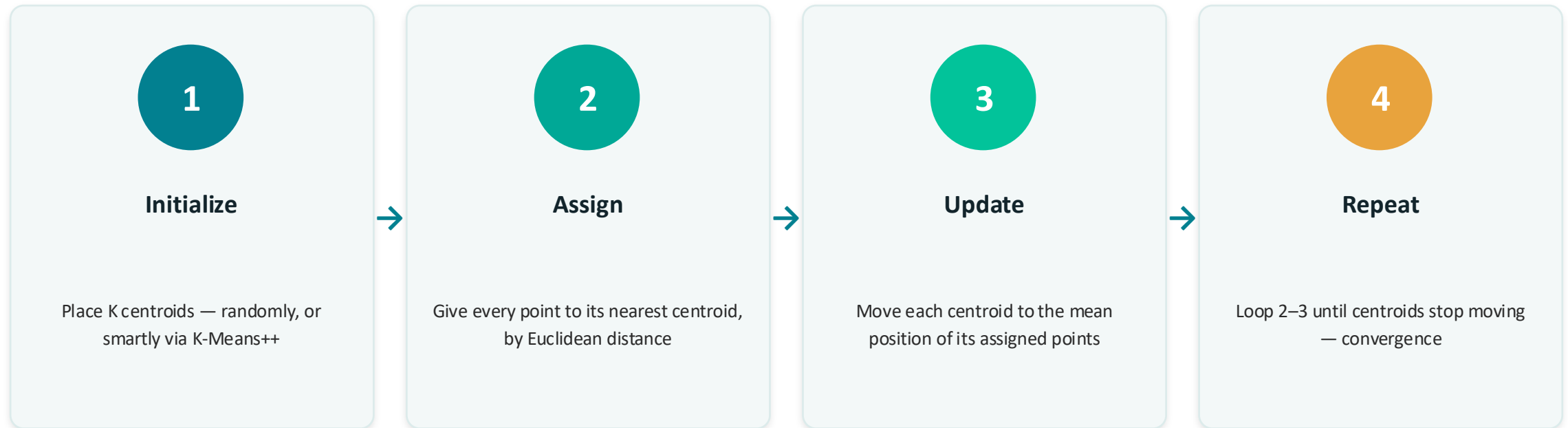
We deliberately choose $K = 4$ as a business-driven compromise

Small enough for a marketing team to build four distinct campaigns around. Large enough to preserve meaningful differences between groups. And — as the persona slides ahead will show — it produces genuinely interpretable results.

This is the single most important judgment call in the whole pipeline — and it's a judgment call, not a formula. When metrics disagree, that's a signal the “right” K depends on what you'll do with the answer.

K-Means, Step by Step (Lloyd's Algorithm)

A loop of “assign → average → assign → average...” until things settle down. No deeper magic — just repeated averaging.



Watch the centroids: they jump to a random starting spot at Iteration 0, then creep toward the true center of each blob. By Iteration 3–4 they've essentially stopped moving — convergence.

Starting Point Matters: Random Init vs. K-Means++

Random Initialization

Picks K random data points as starting centroids. Sometimes you get lucky. Sometimes two centroids start right next to each other and the algorithm settles into a bad local optimum.

Higher inertia
on average

More run-to-run
variance

K-Means++ (sklearn default)

Spreads out the initial centroids on purpose — biasing toward points far from already-chosen centroids. This gives more consistent, usually better, results.

Lower & more
consistent inertia

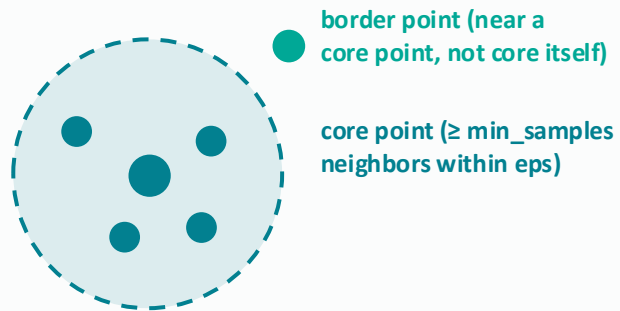
Fewer unlucky
“bad start” runs

That's exactly why `init="k-means++"` is scikit-learn's default — paired with `n_init=10` (rerun 10 times, keep the best result), used throughout this notebook.

DBSCAN — Clustering by Density, Not Distance-to-Center

K-Means assumes clusters are round blobs of similar size. Real customer data often violates that. DBSCAN looks at density instead of centroids — and never needs to be told K in advance.

Core · Border · Noise



Grows clusters by chaining core points close to each other, then attaching border points to whichever chain they touch.

Two parameters, in plain English

eps (ϵ) — the radius of the neighborhood around each point.

min_samples — how many points must be within eps for a point to count as “core.”

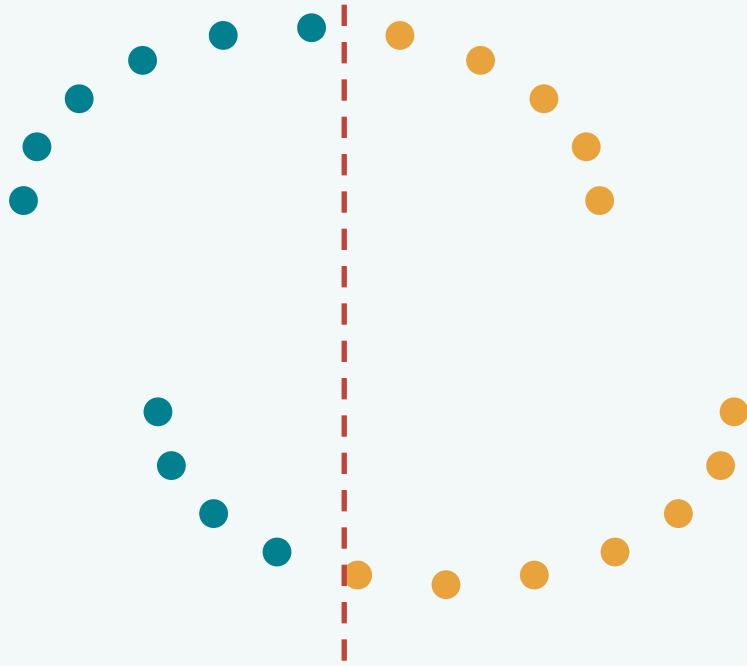
What DBSCAN can do that K-Means can't:

- Find clusters of any shape
- Never needs to be told K in advance
- Flags genuine outliers as noise instead of forcing them into a group

Where K-Means Fails and DBSCAN Shines

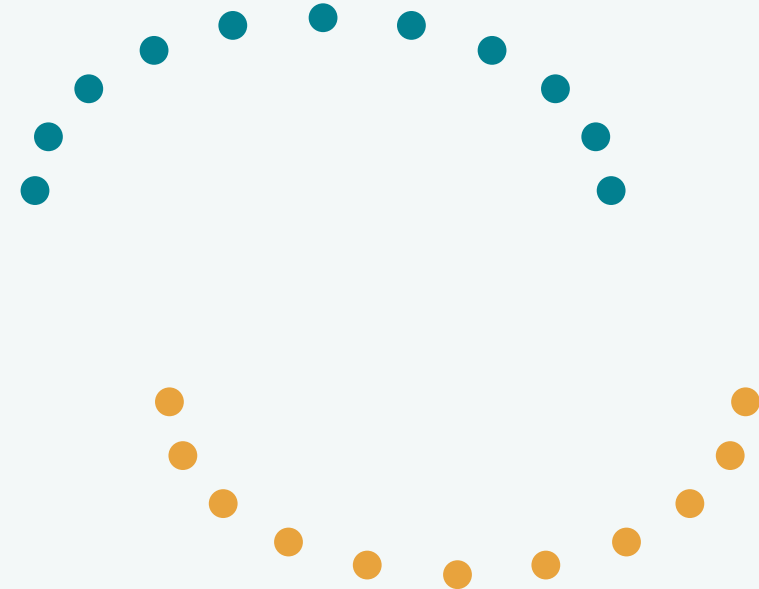
The clearest illustration of why the algorithm you pick matters: two interleaving crescents (“two moons”).

K-MEANS (straight-line boundaries only)



Slices both crescents right down the middle — mixing them regardless of K .

DBSCAN (follows density)



Recovers both crescents perfectly — with zero knowledge that $K=2$ was even right.

Tuning DBSCAN: Finding the Right eps

Too small eps

Almost no point has enough close neighbors → nearly everything becomes noise.

Too large eps

The whole dataset can collapse into one giant cluster.

The systematic fix

A k-distance plot: compute each point's distance to its k-th nearest neighbor ($k = \text{min_samples}$), sort, and look for the elbow.



On our retail data

DBSCAN found 6 clusters + only 17 noise points (1.7%) — sizes almost perfectly even (~140–173 customers each).

Why 6? Our features one-hot-encode 3 categories × 2 genders = 6 combinations. One-hot dummies sit at hypercube corners — each Gender × Category pair forms its own tight, density-separated island.

Two More Tools Worth Knowing

Hierarchical (Agglomerative) Clustering

Starts with every point as its own cluster, then repeatedly merges the two closest clusters until one giant cluster remains.



dendrogram — cut at any height for any number of clusters, after the fact

Extremely useful when you genuinely don't know K and want to see the nested structure before committing.

Gaussian Mixture Models — “Soft” Clustering

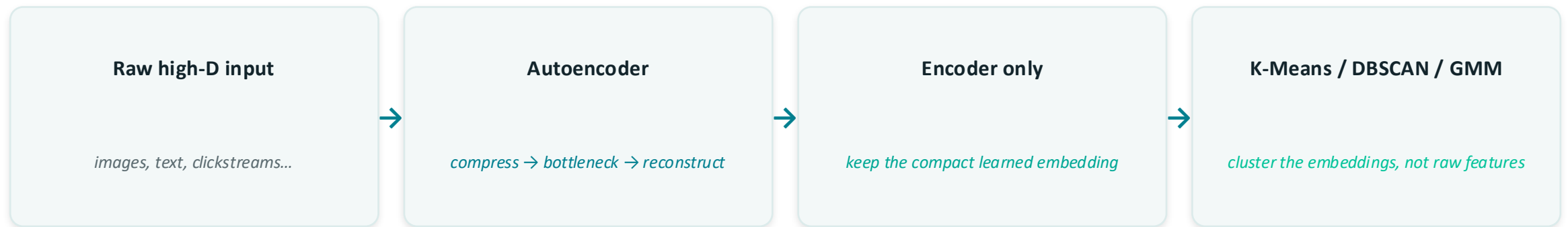
K-Means gives every point a single, hard cluster. GMM assumes overlapping bell-curve distributions and gives every point a probability of belonging to each cluster.

***“70% Occasional Big Spender,
30% High-Value Loyalist”***

A soft, probabilistic label is often more honest than forcing a hard choice — and lets you flag genuinely ambiguous customers for a human to review.

When to Go Deep: Autoencoder-Based Clustering

Rule of thumb: reach for deep learning when the raw feature space is high-dimensional and messy in a way tabular scaling/PCA can't untangle — raw images, free text, clickstreams, hundreds of noisy columns.



The lesson, not a footnote

On this notebook's tabular, 10-dimensional retail features, the deep-learning route needed far more code, more hyperparameters, and a longer training step — for a silhouette score roughly comparable to plain PCA + K-Means. Deep learning earns its complexity when the raw features are genuinely complex. For clean tabular data, PCA + K-Means is simpler, faster, more interpretable, and just as good.

K-Means vs. DBSCAN vs. Deep Learning — Which One?

Roughly know how many segments, likely round/blob-shaped?

K-Means

Don't know K, suspect weird shapes, need outlier detection?

DBSCAN

Everything fits a table with < ~50 meaningful dims after PCA?

K-Means or DBSCAN on PCA features

Raw data is images, text, audio, clickstreams?

Deep-learning embeddings + K-Means / DBSCAN

Want soft, probabilistic membership instead of a hard label?

Gaussian Mixture Model

Want to explore several K values before committing?

Hierarchical Clustering (dendrogram)

Interpretability for a non-technical stakeholder is #1?

K-Means — centroids read as “the average customer”

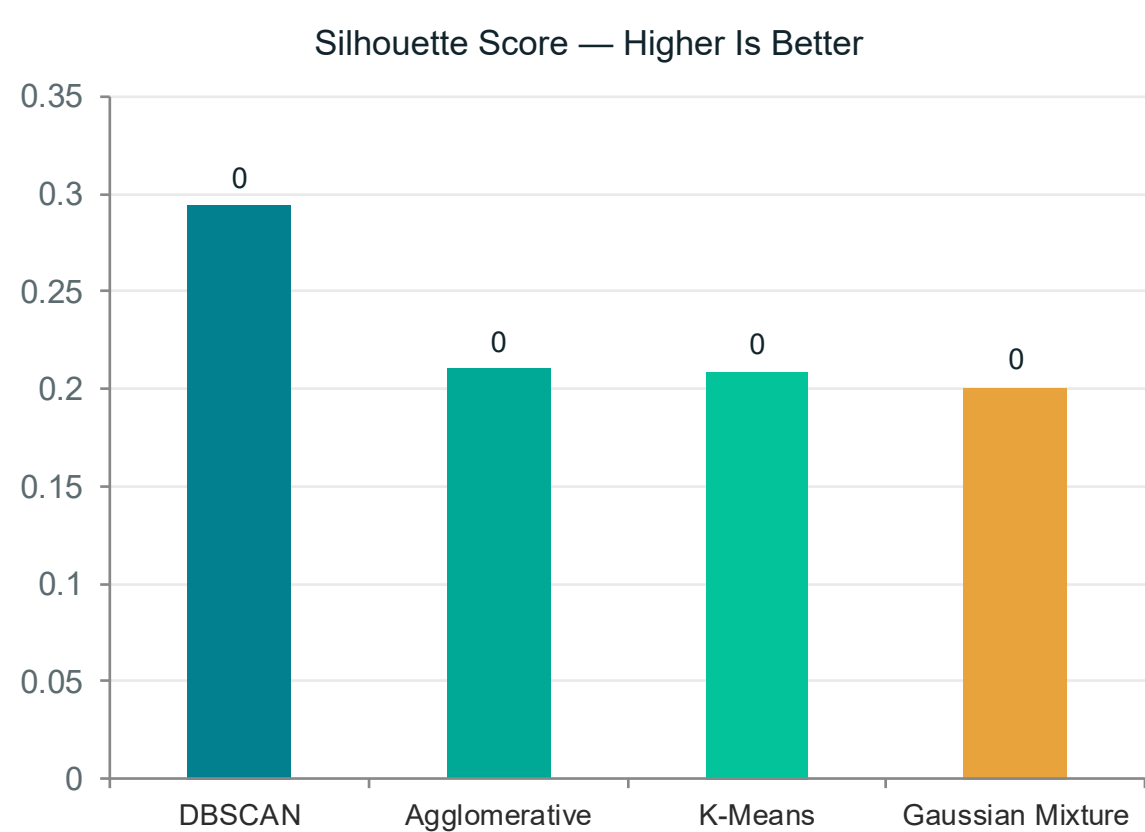
Features are mostly low-cardinality one-hot dummies?

DBSCAN often surprisingly good — corners are density islands

For our retail dataset: DBSCAN actually edges out K-Means on Silhouette and Davies-Bouldin — the Decision Guide is a starting heuristic, not a guarantee. Compute the metrics on your own data (next slide) rather than assuming from theory alone.

The Leaderboard: Comparing Every Algorithm

Same real retail feature set, four algorithms, three core validation metrics, side by side.



	# Clusters	# Noise	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑
DBSCAN	6	17	0.294	1.500	202.3
Agglomerative	4	0	0.210	1.608	197.0
K-Means	4	0	0.209	1.638	189.0
Gaussian Mixture	4	0	0.201	1.832	181.9

DBSCAN posts the best numbers — it's matching the natural Gender × Category corners almost exactly. K-Means gives the simpler K=4 result that's easier to hand to a marketing stakeholder as four named personas.

From Cluster Numbers to Customer Personas

Using our K-Means (K=4) result. First honest observation: % of Customers and % of Revenue are nearly identical across clusters — no strong 80/20 Pareto skew here. “Who buys what” beats “who spends most” as the organizing signal in this data.



Male Beauty Shoppers

Cluster 0 · 141 customers · 14.1%

100% Male · Beauty · Recency 178.6d

ACTION

Bundle grooming kits; target male-specific beauty content — grow basket size in an under-marketed niche.



Female Beauty Shoppers

Cluster 3 · 166 customers · 16.6%

100% Female · Beauty · Recency 188.1d

ACTION

Beauty is naturally repeat-purchase — launch a replenishment subscription or loyalty points program.



Clothing Shoppers

Cluster 1 · 351 customers · 35.1% (largest)

Mixed gender · Clothing · Recency 194.9d (highest!)

ACTION

Best win-back target: send a lapsed-customer discount for new seasonal arrivals.



Electronics Shoppers

Cluster 2 · 342 customers · 34.2%

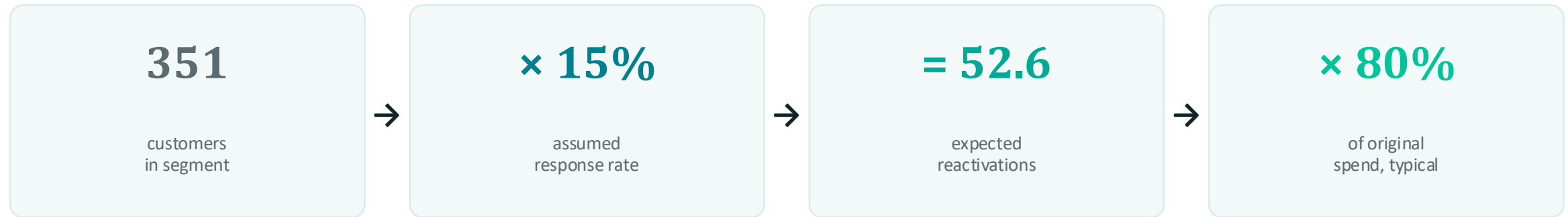
Mixed gender · Electronics · Recency 170.0d (most recent)

ACTION

Currently engaged — upsell warranties/accessories, time offers around new launches.

The Money Slide: A Win-Back Campaign, Priced Out

A cluster profile is only useful once someone can answer: “okay, so what happens to our revenue if we act on this?” Target: Cluster 1 (Clothing Shoppers) — our highest-recency, largest segment.



EXPECTED RECOVERED REVENUE

\$18,668

≈ 4.09% of this dataset's total revenue — from one targeted email campaign to one segment.

Every technique from this deck fed this one number:

scaling → PCA → choosing K → K-Means → honest cluster profiling → one clean, defensible business estimate. Swap the assumed response rate or repeat-spend ratio to run your own what-if scenarios.

Key Takeaways

- **Structure → story → action**

Unsupervised learning finds structure with no labels; the payoff is entirely in how well you translate it.

- **The Curse is real & measurable**

Distances collapse as dimensions grow — exactly why PCA earns its place in the pipeline.

- **K-Means++ beats random init**

Lower and more consistent inertia across runs — always prefer it.

- **Hierarchical & GMM round out the kit**

Dendrograms to explore K visually; GMM for honest, probabilistic membership.

- **Scaling is mandatory**

An unscaled Monetary column will silently hijack your entire clustering result.

- **K is a multi-metric decision**

Elbow, Silhouette, Davies-Bouldin, Calinski-Harabasz each tell part of the story; business sense breaks ties.

- **DBSCAN does what K-Means can't**

Non-convex shapes and genuine outlier detection, at the cost of tuning `eps` / `min_samples`.

- **Deep learning is a tool, not a default**

It earns its complexity on genuinely high-dimensional, unstructured data — not clean tabular features.

The single most-skipped step: translate cluster centroids into personas, and personas into a dollar-and-cents action plan.

Your Turn: Go Re-Run the Pipeline

1

New feature set

Drop the one-hot Gender/Category columns; cluster only on Recency, Monetary, Quantity, AvgItemPrice, Age. Does the best K change? Do personas still make sense?

2

Tune DBSCAN properly

Use the k-distance elbow plot to try 2–3 eps values and 2–3 min_samples values. Report the best silhouette (excluding noise) and explain why.

3

Your own what-if business case

Pick a different cluster (e.g. highest-value instead of at-risk) and design a different action (e.g. loyalty program). State your assumed response rate and show the revenue impact.

4

Bonus

Add a 5th feature of your own design (e.g. “is this a weekend purchase?”) and see whether it changes which customers get grouped together.